Introduction to Blockchain

CSE-355

Ahmed Akhtar
Department of Computer Science
SMCS, IBA Karachi
Spring 2026

Lecture 11
Tuesday, Feb 24, 2026

Bitcoin Transaction (legacy)

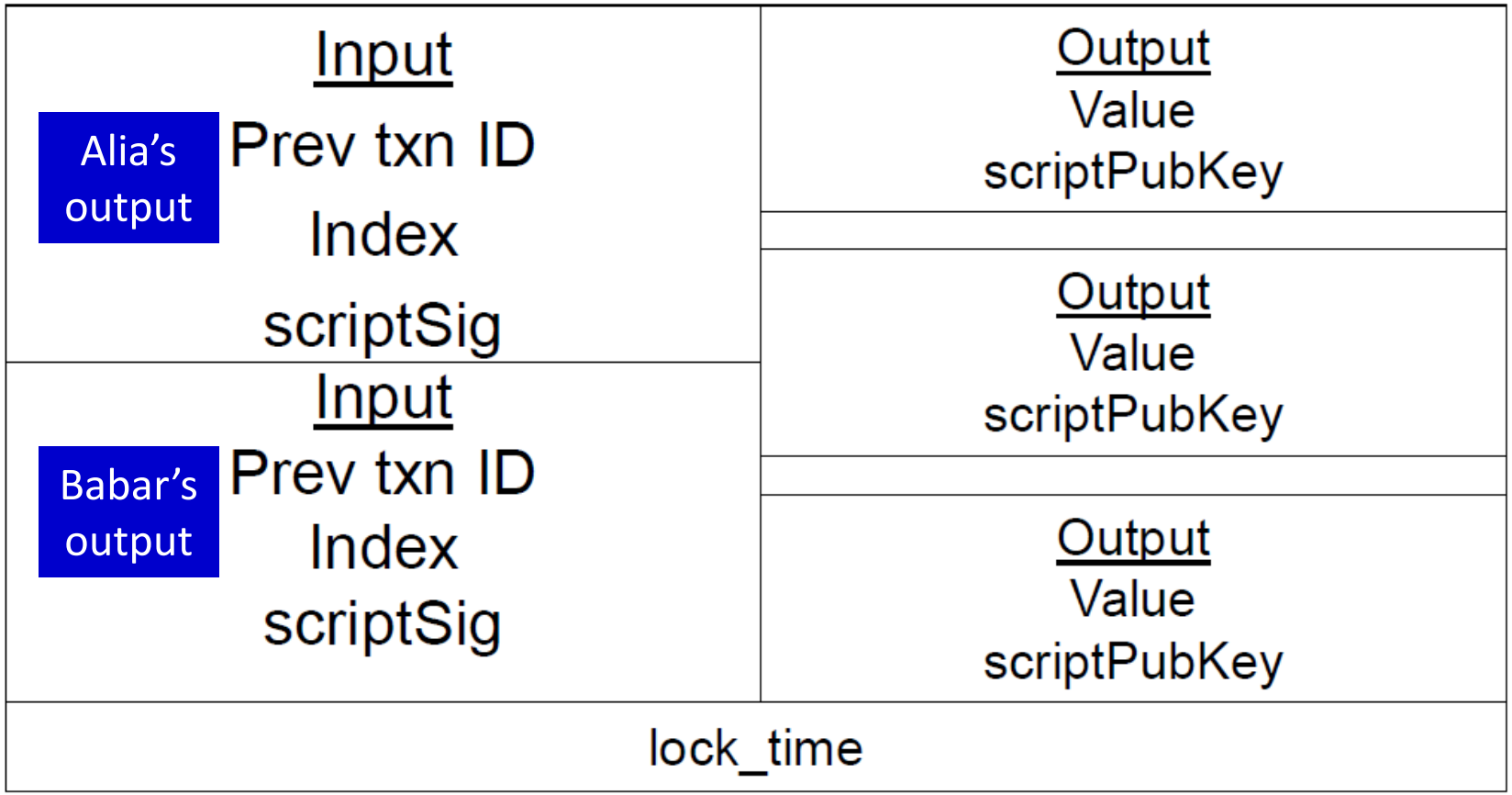
Transaction ID:

7cc90809ea6c90899086ed8607eea8dcc1bbe2ee3fbaa662f3f597888575c4c9

	OUTPUT
If ScriptPubKey(ScriptSig) **then** create output If transaction inputs are valid then	Index 0 Value: 0.00025302 BTC scriptPubKey: OP_DUP OP_HASH160 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515 OP_EQUALVERIFY OP_CHECKSIG Lock to Babar's PubKey Hash Index 1 Value: 0.01094000 BTC scriptPubKey: OP_DUP OP_HASH160 9772ba84d827ac93d719a2a40f3f5dcb8963b8ad OP_EQUALVERIFY OP_CHECKSIG Lock to Alia's PubKey Hash

Bitcoin Transaction (legacy)

Multiple inputs and outputs



Bitcoin Transaction (legacy)

Unspendable Output

INPUT	OUTPUT
Prev. Trans. ID: Index: 0 ScriptSig	Index 0 Value: 0 BTC **scriptPubKey**: **OP_Return** **<any thing>** Unspendable Output
	Index 1 Value: 0.00025302 BTC **scriptPubKey**: OP_DUP OP_HASH160 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515 OP_EQUALVERIFY OP_CHECKSIG

Bitcoin Transaction (legacy)

INPUT	OUTPUT
Prev. Trans. ID: 0f07ec1c3738873cd6336f0e2c92094bf0 8d0ad3bce8aba0efa2eb9d172fd60c Index: 0 ScriptSig 304402208a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993 b6f6b9c5b1f3a2d702202f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41 d6e2a2b9df62d4a056c6bb6f23ef01 **(Signature)** 026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5 e680b2ab2cf8 **(Public Key)**	Index 0 Value: 0.00025302 BTC scriptPubKey: OP_DUP OP_HASH160 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515 OP_EQUALVERIFY OP_CHECKSIG Index 1 Value: 0.01094000 BTC scriptPubKey: OP_DUP OP_HASH160 9772ba84d827ac93d719 a2a40f3f5dcb8963b8ad OP_EQUALVERIFY OP_CHECKSIG

Transaction ID:

7cc90809ea6c90899086ed8607eea8dcc1bbe2ee3fbaa662f3f597888575c4c9

Bitcoin Transaction (legacy)

scriptSig

- Signature is created using Elliptic Curve Digital Signature Algorithm (ECDSA)
- ECDSA signature is a pair (r, s)
  - r and s are both integers
- ECDSA signature format
  - **30** <total length> **02** <r length> <r bytes> **02** <s length> <s bytes> <sighash byte>

Bitcoin Transaction (legacy)

ECDSA signature format

- **30** <total length> **02** <r length> <r bytes> **02** <s length> <s bytes> <sighash byte>
  - 30 - DER (Distinguished Encoding Rules) sequence
  - <total length> - Total length in bytes of everything that follows
  - 02 – Integer tag
  - <r length> - Length in bytes of integer r
  - <r bytes> - The actual bytes of r
  - 02 – Integer tag
  - <s length> - Length in bytes of integer s
  - <s bytes> - The actual bytes of s
  - <sighash byte> - 1 byte flag indicating which parts of the transaction were hashed and signed
    - 0x01 – all inputs and all outputs hashed and signed
    - 0x02 – all inputs are hashed and signed but no output

Bitcoin Transaction (legacy)

ECDSA signature format

- **30** <total length> **02** <r length> <r bytes> **02** <s length> <s bytes> <sighash byte>
  - 30 - DER (Distinguished Encoding Rules) sequence
  - <total length> - 0x44 (68 bytes)
  - 02 – Integer tag
  - <r length> - 0x20 (32 bytes)
  - <r bytes> - 8a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d7
  - 02 – Integer tag
  - <s length> - 0x20 (32 bytes)
  - <s bytes> - 2f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef
  - <sighash byte> - 0x01

3044**02**208a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d7**02**2
02f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef01

Bitcoin Transaction (legacy)

ECDSA signature format

- **30** <total length> **02** <r length> <r bytes> **02** <s length> <s bytes> <sighash byte>
  - 30 - DER (Distinguished Encoding Rules) sequence
  - <total length> - 0x45 (69 bytes)
  - 02 – Integer tag
  - <r length> - 0x21 (33 bytes)
  - <r bytes> - 008a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d7
  - 02 – Integer tag
  - <s length> - 0x20 (32 bytes)
  - <s bytes> - 2f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef
  - <sighash byte> - 0x01

3045**02**2100 8a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d7**0**
2202 f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef01

Bitcoin Transaction (legacy)

INPUT	OUTPUT
Prev. Trans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c Index: 0 ScriptSig 304402208a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d702202f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef01 **(Signature)** 026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8 **(Public Key)**	Index 0 Value: 0.00025302 BTC scriptPubKey: OP_DUP OP_HASH160 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515 OP_EQUALVERIFY OP_CHECKSIG Index 1 Value: 0.01094000 BTC scriptPubKey: OP_DUP OP_HASH160 9772ba84d827ac93d719a2a40f3f5dcb8963b8ad OP_EQUALVERIFY OP_CHECKSIG

Transaction ID:

7cc90809ea6c90899086ed8607eea8dcc1bbe2ee3fbaa662f3f597888575c4c9

Bitcoin Transaction (legacy)

INPUT	OUTPUT
Prev. Trans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c Index: 0 ScriptSig 3045022100 8a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d702202f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef01 **(Signature)** 026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8 **(Public Key)**	Index 0 Value: 0.00025302 BTC scriptPubKey: OP_DUP OP_HASH160 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515 OP_EQUALVERIFY OP_CHECKSIG Index 1 Value: 0.01094000 BTC scriptPubKey: OP_DUP OP_HASH160 9772ba84d827ac93d719a2a40f3f5dcb8963b8ad OP_EQUALVERIFY OP_CHECKSIG

Transaction ID:

~~7cc90809ea6c90899086ed8607eea8dcc1bbe2ee3fbaa662f3f597888575c4c9~~

Bitcoin Transaction (legacy)

INPUT	OUTPUT
Prev. Trans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c Index: 0 ScriptSig 3045022100 8a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d702202f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef01 **(Signature)** 026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8 **(Public Key)**	Index 0 Value: 0.00025302 BTC scriptPubKey: OP_DUP OP_HASH160 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515 OP_EQUALVERIFY OP_CHECKSIG Index 1 Value: 0.01094000 BTC scriptPubKey: OP_DUP OP_HASH160 9772ba84d827ac93d719a2a40f3f5dcb8963b8ad OP_EQUALVERIFY OP_CHECKSIG

Transaction ID:

~~7cc90809ea6c90899086ed8607eea8dcc1bbe2ee3fbaa662f3f597888575c4c9~~

7b9ab087c86818551aa77f59bb2c35bd774abed7fcb19c7e840b502b2604271c

Bitcoin Transaction (legacy)

Transaction malleability

- A transaction's ID (**txid**) can be changed *without altering its actual effect* on the blockchain
  - The txid is computed as the **hash of the serialized transaction**
  - In legacy Bitcoin, this serialization included the **signatures** inside the scriptSig
  - **Problem:** ECDSA signatures allow multiple valid DER encodings (e.g., adding/removing leading zeros in r or s)
  - Both encodings are valid but produce different **txids**
- An attacker can slightly modify the signature's byte format, changing the txid — even though the transaction still spends the same inputs and creates the same outputs

Why Transaction Malleability is an issue?

- Unreliable Transaction IDs
  - Wallets track transactions by txid
  - If the txid can be changed, wallets may not recognize that a confirmed transaction is the same one they sent
- Double-Spend Risks
  - Attackers can alter a pending transaction’s signature, rebroadcast it, and miners might confirm the modified version instead of the original
- Network Confusion
  - Nodes and users may see multiple versions of the same transaction with different IDs, creating uncertainty until one is mined

Why Transaction Malleability is an issue?

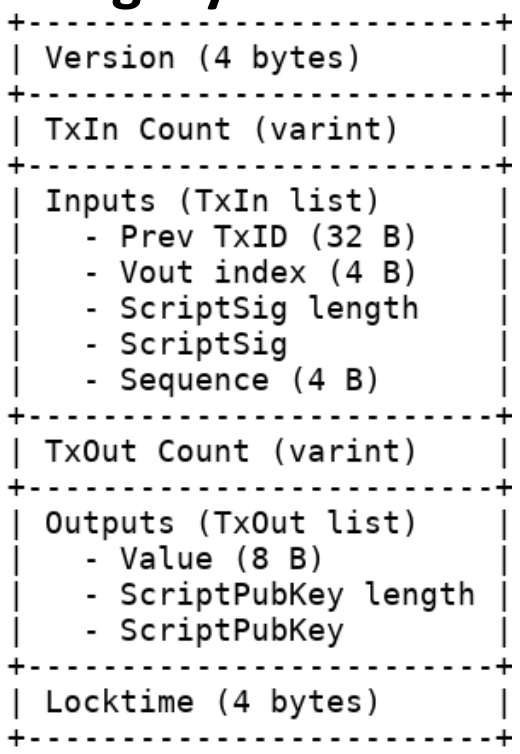
- Breaks Layer-2 Protocols
  - Off-chain protocols (e.g., early payment channels, Lightning prototypes) depend on knowing exact txids
  - Malleability makes it impossible to pre-build transactions that rely on previous txids

How to prevent transaction malleability?

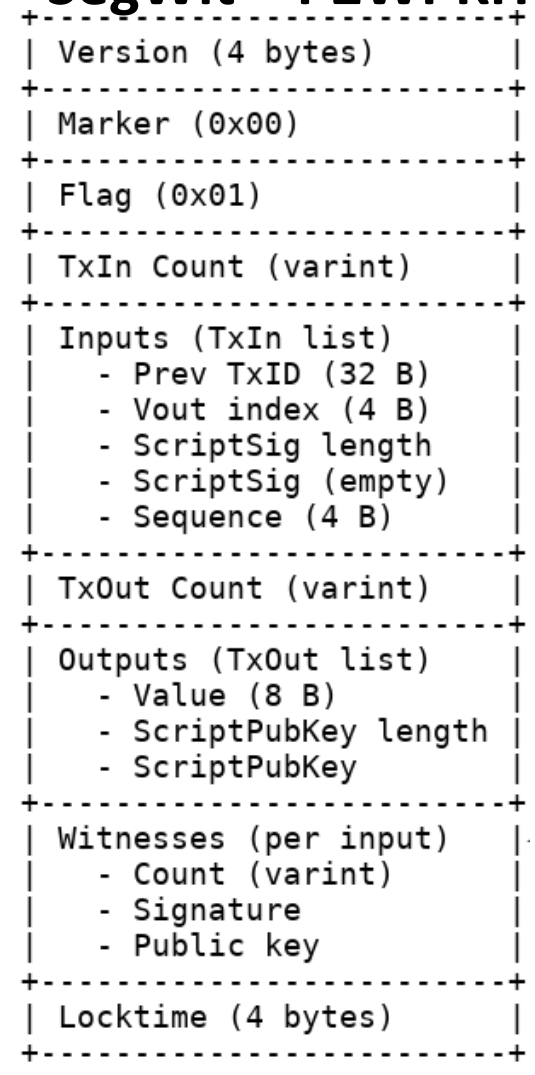
- scriptSig = <ECDSA signature, Public key> acts as a witness proving that:
  - The payer has authorized this transaction
  - The payer can unlock the unspent output of previous transaction
- To prevent transaction malleability, we need to segregate this witness from the transaction input
  - Segregated Witness (SegWit) transaction
  - Txid is not created using scriptSig

Bitcoin – legacy vs. SegWit Transactions

Legacy – P2PKH



SegWit – P2WPKH



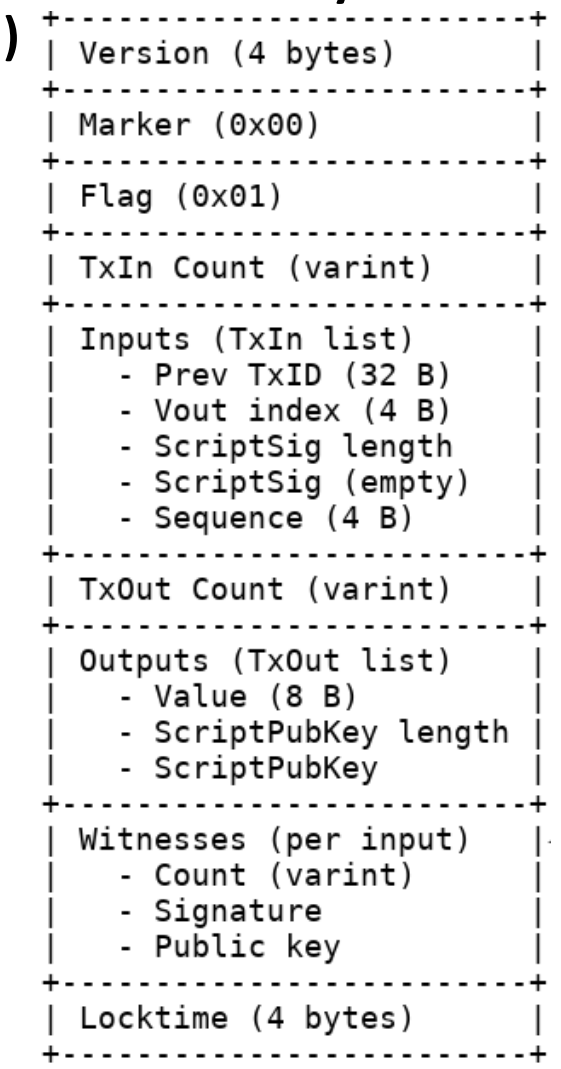
SegWit Transaction

Version: 01000000 --- Big Endian encoding
Marker: 00
Flag: 01
TxInCount: 01 --1 input

Prev TXID: 7cc90809ea6…2f3f597888575c4c9
Vout index: 00000000
scriptSig length: 00
scriptSig: “” --1 empty
Sequence: ffffffff

TxOutCount: 01 --1 output
Value: 0000000000809698 -- 0.1 BTC (10^7 satoshis)
ScriptPubKey len: 16 -- 22 bytes
ScriptPubKey: 00 14 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515
<Segwit Version 0> <length of witness program> <20 byte pubkeyHash>
Witness for each input
Count: 02 --No. of items in witness stack
Signature: 30450221008a09e17ddf7f3e7d134567890099fe42….ef01
Public Key: 026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8

Pay to Witness Public Key Hash (P2WPKH)



SegWit Transaction – P2WPKH

INPUT	OUTPUT
Prev. Trans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c Index: 0 ScriptSig ""	Index 0 Value: 0.00025302 BTC scriptPubKey: Babar 00 14 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515 Index 1 Value: 0.01094000 BTC scriptPubKey: Alia 00 14 92fd4e32824b531e5152fa9c4b9585b74fd5fd8c

Witness

Signature:
304402208a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d702202f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef01

Public Key: Alia
026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8

SegWit Transaction – P2WPKH

INPUT

Prev. Trans. ID:
0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c
Index: 0

ScriptSig ""

Bitcoin internally expands the scriptPubKey of Prev. Trans. Output into the "virtual" script

OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG

OUTPUT

Index 0
Value: 0.00025302 BTC

No OP Codes!

scriptPubKey:
00 14 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515

Index 1
Value: 0.01094000 BTC

scriptPubKey:
00 14 92fd4e32824b531e5152fa9c4b9585b74fd5fd8c

Witness

Signature:
304402208a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d702202f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef01

Public Key:
026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8

Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de
78f6844a548d548b0d27634c02207b127fa009b9d755b84
8772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6
a4304a5e680b2ab2cf8 **Alia’s public key**

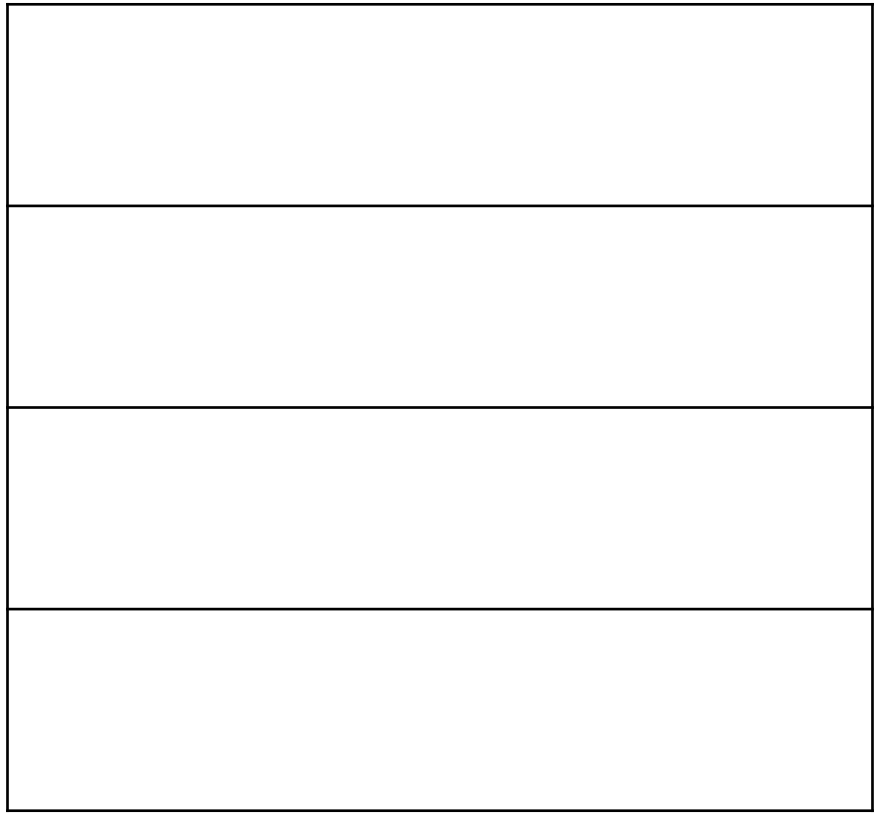
ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

SegWit transaction!

Expand ScriptPubKey to virtual scriptPubkey for P2WPKH

Stack



Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de
78f6844a548d548b0d27634c02207b127fa009b9d755b84
8772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6
a4304a5e680b2ab2cf8 **Alia’s public key**

ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

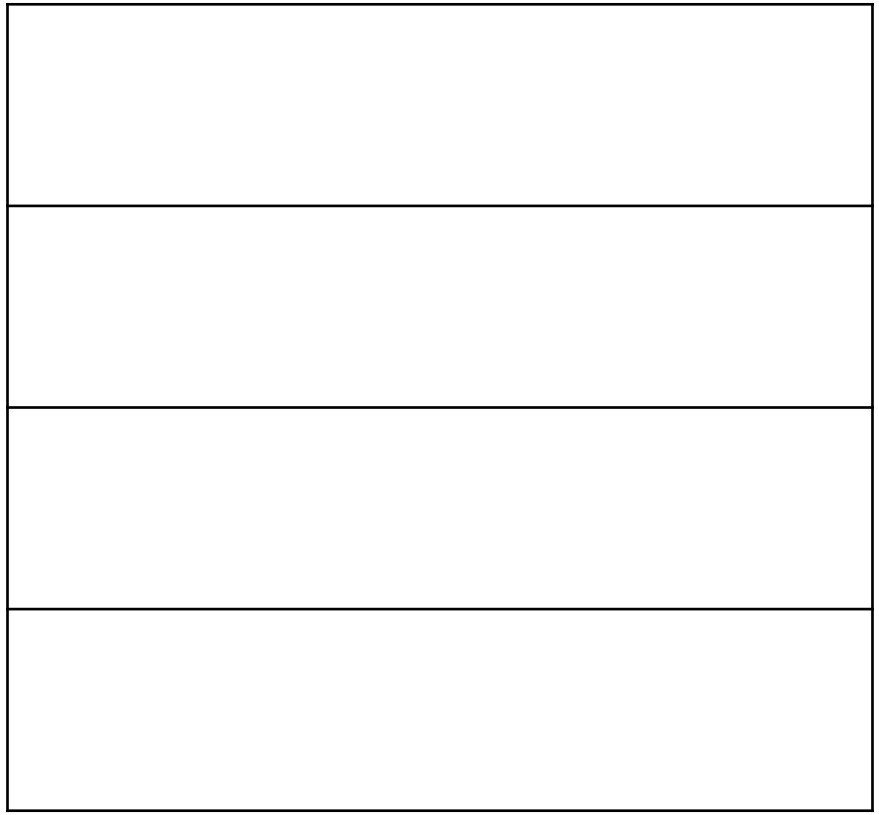
Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475af
f76e>
OP_EQUALVERIFY
OP_CHECKSIG

SegWit transaction!

Expand ScriptPubKey to virtual scriptPubkey for P2WPKH

Stack



Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de
78f6844a548d548b0d27634c02207b127fa009b9d755b84
8772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6
a4304a5e680b2ab2cf8 **Alia’s public key**

ScriptPubKey: (Prev. Trans.)

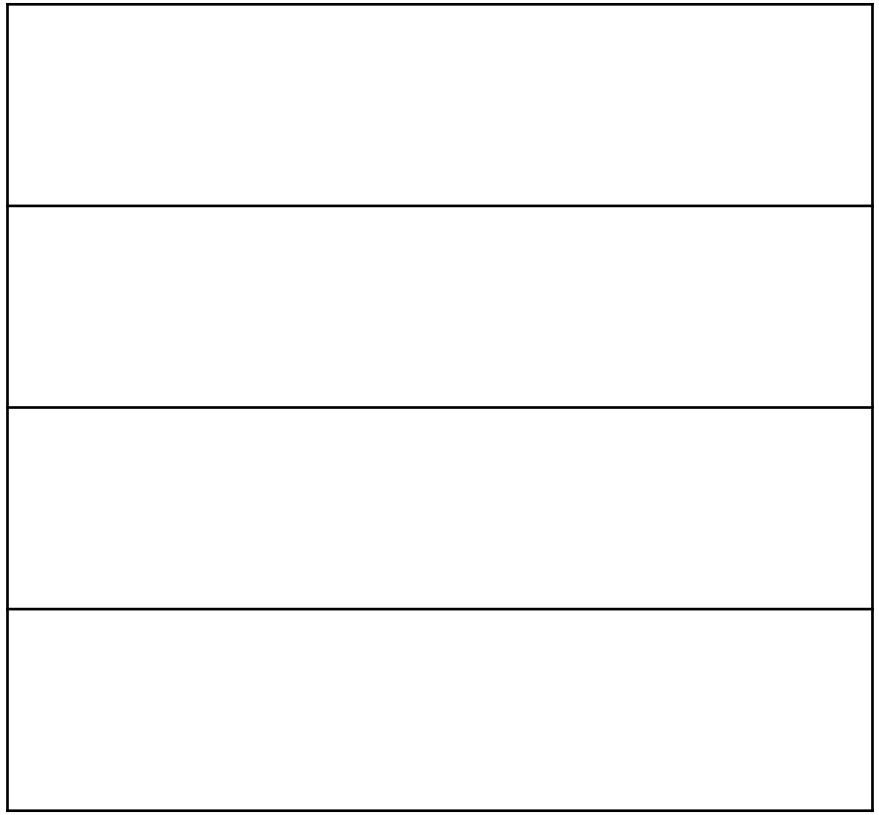
00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475af
f76e>
OP_EQUALVERIFY
OP_CHECKSIG

Push witness items to stack

Stack



Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de
78f6844a548d548b0d27634c02207b127fa009b9d755b84
8772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6
a4304a5e680b2ab2cf8 **Alia’s public key**

ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475af
f76e>
OP_EQUALVERIFY
OP_CHECKSIG

Push witness items to stack

Stack

026a7cd9a8f65b260bd459d8c1e823110a 699391cbaabb6a4304a5e680b2ab2cf8
304402205e4a6ac74d35de1ffc1188f7172 73bada586b3de78f6844a548d548b0d276 34c02207b127fa009b9d755b848772dcee 21c4a05554501fd5e0d8a60ee8b9f9d13d a7d01

Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8 **Alia’s public key**

ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475aff76e>
OP_EQUALVERIFY
OP_CHECKSIG

Execute OP Codes!

Stack

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8
304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8 **Alia’s public key**

ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475aff76e>
OP_EQUALVERIFY
OP_CHECKSIG

Stack

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8
026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8
304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8 **Alia’s public key**

ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475aff76e>
OP_EQUALVERIFY
OP_CHECKSIG

Stack

76960c3d0cdf8076ecd0cc220022384475aff76e
026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8
304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8 **Alia’s public key**

ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475aff76e>
OP_EQUALVERIFY
OP_CHECKSIG

Stack

76960c3d0cdf8076ecd0cc220022384475aff76e
76960c3d0cdf8076ecd0cc220022384475aff76e
026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8
304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8 **Alia’s public key**

ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475aff76e>
OP_EQUALVERIFY
OP_CHECKSIG

Stack

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8
304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8 **Alia’s public key**

ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475aff76e>
OP_EQUALVERIFY
OP_CHECKSIG

Stack

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8
304402205e4a6ac74d35de1ffc1188f717273bada586b3de78f6844a548d548b0d27634c02207b127fa009b9d755b848772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Execution – P2WPKH

ScriptSig: “ ”

Witness:

304402205e4a6ac74d35de1ffc1188f717273bada586b3de
78f6844a548d548b0d27634c02207b127fa009b9d755b84
8772dcee21c4a05554501fd5e0d8a60ee8b9f9d13da7d01

Alia’s Signatures

026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6
a4304a5e680b2ab2cf8 **Alia’s public key**

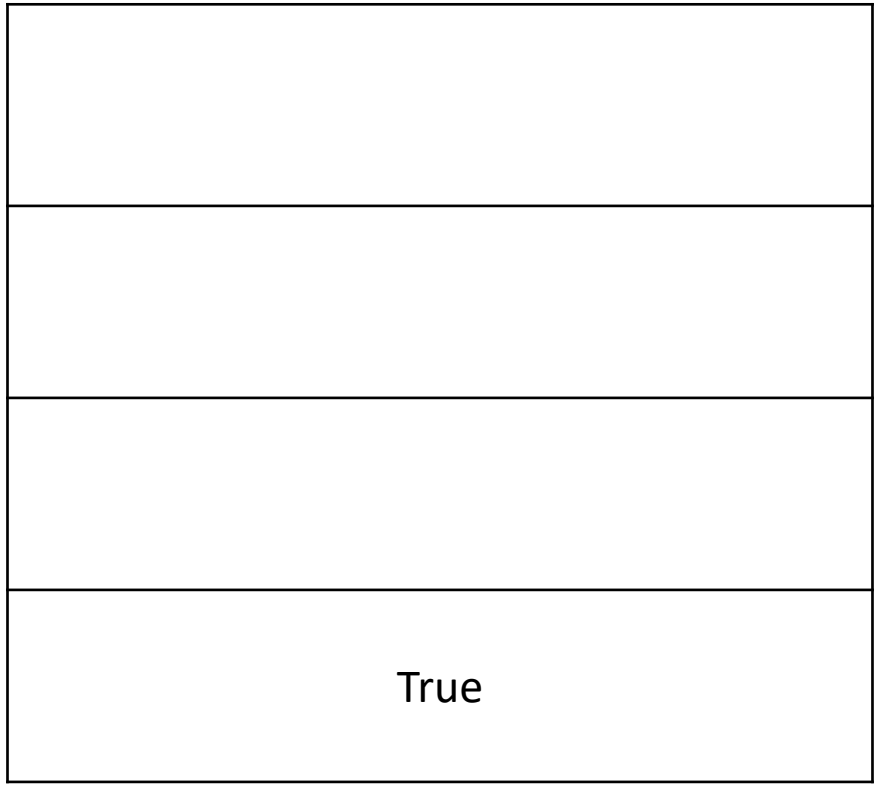
ScriptPubKey: (Prev. Trans.)

00 14 076960c3d0cdf8076ecd0cc220022384475aff76e

Virtual ScriptPubKey

OP_DUP
OP_HASH160
<076960c3d0cdf8076ecd0cc220022384475af
f76e>
OP_EQUALVERIFY
OP_CHECKSIG

Stack



SegWit Transaction – P2WPKH

INPUT	OUTPUT
Prev. Trans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c Index: 0 ScriptSig ""	Index 0 Value: 0.00025302 BTC scriptPubKey: Locked to Babar 00 14 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515 Index 1 Value: 0.01094000 BTC scriptPubKey: Locked to Alia 00 14 92fd4e32824b531e5152fa9c4b9585b74fd5fd8c

Witness

Signature:
304402208a09e17ddf7f3e7d1d6e18b3b42f9e445c8c6f5f2e93f993b6f6b9c5b1f3a2d702202f7d8b03c2f1763e8c5ff0b8a0b6c80b7d41d6e2a2b9df62d4a056c6bb6f23ef01

Public Key: Alia
026a7cd9a8f65b260bd459d8c1e823110a699391cbaabb6a4304a5e680b2ab2cf8

Lecture 12
Thursday, Feb 26, 2026

Multisig Transaction

Multisig – Multiple signatures

- A Bitcoin transaction that requires M-of-N keys to authorize spending
  - Example: 2-of-3 multisig → any 2 signatures out of 3 keys are needed

Why?

- ***Security***: Protects funds even if one key is lost or stolen
- ***Shared Accounts***: Company treasuries or family accounts requiring multiple approvals
- ***Escrow Services***: Buyer, Seller, and Arbitrator in 2-of-3 setups
- ***Lightning Network***: Payment channels are built on 2-of-2 multisig outputs

Multisig Transaction – How it works?

1. Locking (Output Side)

- Bitcoin is sent to a **script**:
  - **OP_2** <PubKey1> <PubKey2> <PubKey3> **OP_3 OP_CHECKMULTISIG**
- This script is hashed into an address (P2SH or P2WSH)

2. Unlocking (Input Side)

- To spend, the spender provides:
  - M valid signatures (e.g., 2 signatures in a 2-of-3)
  - The witness script proving the locking conditions

3. Verification

- Bitcoin validates: Do the provided signatures match the required keys?

Multisig Transaction - Format

SegWit Multisig Transaction Format (P2WSH)

```
SegWit Transaction – P2WSH Multisig

Version (4 bytes)

Marker (0x00)
Flag (0x01)

TxIn Count (varint)

Inputs (TxIn list):
  - Prev TxID (32 B)
  - Vout index (4 B)
  - ScriptSig length (0)
  - ScriptSig (empty)
  - Sequence (4 B)

TxOut Count (varint)

Outputs (TxOut list):
  - Value (8 B)
  - ScriptPubKey length
  - ScriptPubKey:
      0x00 0x20 <32-byte SHA256(witnessScript)>

Witnesses (per input):
  - Count (varint)
  - OP_0 (due to CHECKMULTISIG bug)
  - M signatures
  - Witness script (redeem script):
      OP_M <pubkeys...> OP_N OP_CHECKMULTISIG

Locktime (4 bytes)
```

Multisig Transaction – Execution

INPUT	OUTPUT
Prev. Trans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c Multisig trans. Index: 0	Index 0 Value: 0.00025302 BTC scriptPubKey: 00 14 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515

Witness

OP_0
<sigA>304402208a09e17ddf7f3e7….
<sigC>d1d6e18b3b42f9e445c8c6f….
<witnessScript> OP_2 <PubKeyA> <PubKeyB> <PubKeyC> OP_3 OP_CHECKMULTISIG

Multisig Transaction – Execution

INPUT	OUTPUT
Prev. Trans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c Multisig trans. Index: 0	Index 0 Value: 0.00025302 BTC scriptPubKey: 00 14 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515

Tans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c

Output:
Index 0
Value: 00026102
ScriptPubKey:
0020 0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef

32-byte hash value follows – P2WSH

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

The last witness item must contain the witness script

Stack

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Compute SHA256(<witnessScript>) and check if this is equal to the 32-byte value in the scriptPubkey of the Prev. Trans

If not equal → fail (transaction invalid)

If equal → continue

Tans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c

Output:

Index 0

Value: 00026102

ScriptPubKey:

00 20 0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Push all items in the witness on the stack except the last item, which is the witness script

Stack

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

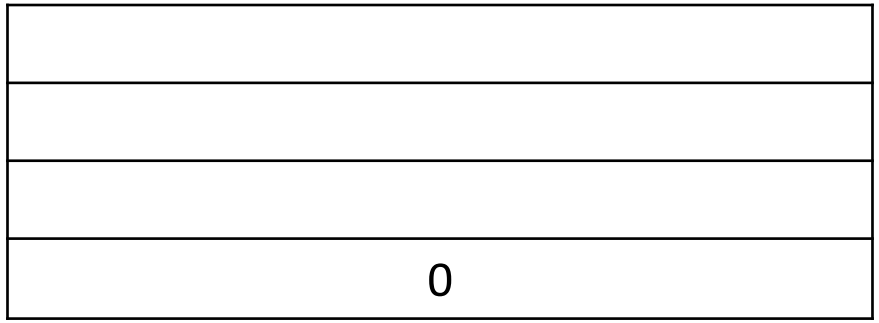
<PubKeyC>

OP_3

OP_CHECKMULTISIG

Push all items in the witness on the stack except the last item, which is the witness script

Stack



Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Push all items in the witness on the stack except the last item, which is the witness script

Stack

<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0
<sigA>304402208a09e17ddf7f3e7….
<sigC>d1d6e18b3b42f9e445c8c6f….
<witnessScript>
    OP_2
    <PubKeyA>
    <PubKeyB>
    <PubKeyC>
    OP_3
    OP_CHECKMULTISIG

Push all items in the witness on the stack except the last item, which is the witness script

Stack

<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Execute the witness script

Stack

<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Execute the witness script

Stack

2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0
<sigA>304402208a09e17ddf7f3e7….
<sigC>d1d6e18b3b42f9e445c8c6f….
<witnessScript>
    OP_2
    <PubKeyA>
    <PubKeyB>
    <PubKeyC>
    OP_3
    OP_CHECKMULTISIG

Execute the witness script

Stack

<PubKeyA>
2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Execute the witness script

Stack

<PubKeyB>
<PubKeyA>
2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Execute the witness script

Stack

<PubKeyC>
<PubKeyB>
<PubKeyA>
2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Execute the witness script

Stack

3
<PubKeyC>
<PubKeyB>
<PubKeyA>
2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Execute the witness script

Stack

3
<PubKeyC>
<PubKeyB>
<PubKeyA>
2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Pop

n = 3

Stack

<PubKeyC>
<PubKeyB>
<PubKeyA>
2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Pop 3 public keys

n = 3

<PubKeyC>

<PubKeyB>

<PubKeyA>

Stack

2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Pop

n = 3

<PubKeyC>

<PubKeyB>

<PubKeyA>

m=2

Stack

<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….
0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

Pop 2 signatures

n = 3

<PubKeyC>

<PubKeyB>

<PubKeyA>

m=2

<sigC>d1d6e18b3b42f9e445c8c6f….

<sigA>304402208a09e17ddf7f3e7….

Stack

0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

n = 3

<PubKeyC>

<PubKeyB>

<PubKeyA>

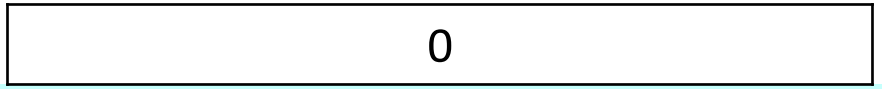
m=2

<sigC>d1d6e18b3b42f9e445c8c6f….

<sigA>304402208a09e17ddf7f3e7….

verify(sigC, [A,B,C]) → valid (matches C)

Stack



Mutisig Execution – P2WSH

Witness:

OP_0
<sigA>304402208a09e17ddf7f3e7….
<sigC>d1d6e18b3b42f9e445c8c6f….
<witnessScript>
    OP_2
    <PubKeyA>
    <PubKeyB>
    <PubKeyC>
    OP_3
    OP_CHECKMULTISIG

n = 3
<PubKeyC>
<PubKeyB>
<PubKeyA>
m=2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….

verify(sigC, [A,B,C]) → valid (matches C)
verify(sigA, [A,B,C]) → valid (matches A)

Stack

0

Mutisig Execution – P2WSH

Witness:

OP_0
<sigA>304402208a09e17ddf7f3e7….
<sigC>d1d6e18b3b42f9e445c8c6f….
<witnessScript>
    OP_2
    <PubKeyA>
    <PubKeyB>
    <PubKeyC>
    OP_3
    OP_CHECKMULTISIG

n = 3
<PubKeyC>
<PubKeyB>
<PubKeyA>
m=2
<sigC>d1d6e18b3b42f9e445c8c6f….
<sigA>304402208a09e17ddf7f3e7….

verify(sigC, [A,B,C]) → valid (matches C)
verify(sigA, [A,B,C]) → valid (matches A)
count(valid) = 2 = m → success

Stack

0

Mutisig Execution – P2WSH

Witness:

OP_0

<sigA>304402208a09e17ddf7f3e7….

<sigC>d1d6e18b3b42f9e445c8c6f….

<witnessScript>

OP_2

<PubKeyA>

<PubKeyB>

<PubKeyC>

OP_3

OP_CHECKMULTISIG

verify(sigC, [A,B,C]) → valid (matches C)

verify(sigA, [A,B,C]) → valid (matches A)

count(valid) = 2 = m → success

POP 0 (CHECKMULTISIG bug), then PUSH TRUE

Stack

TRUE

Multisig Transaction – Execution

INPUT	OUTPUT
Prev. Trans. ID: 0f07ec1c3738873cd6336f0e2c92094bf08d0ad3bce8aba0efa2eb9d172fd60c Multisig trans. Index: 0	Index 0 Value: 0.00025302 BTC scriptPubKey: 00 14 2fa9c4b9585b74fd5fd8c92fd4e32824b531e515

Witness

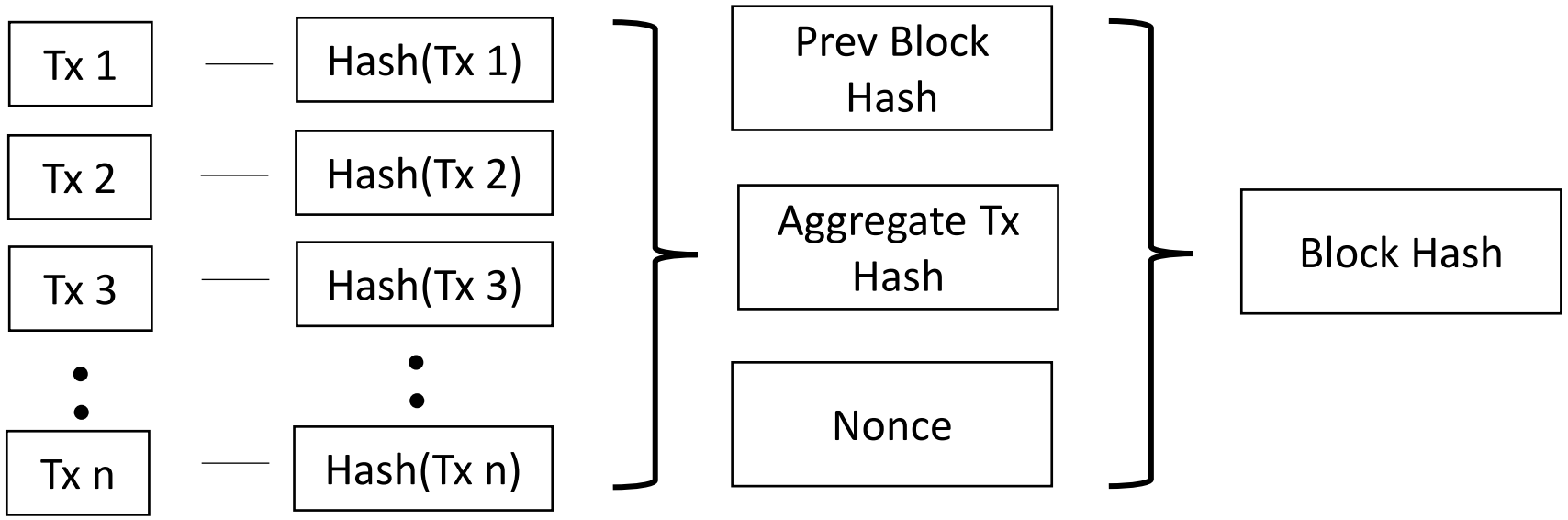
OP_0
<sigA>304402208a09e17ddf7f3e7….
<sigC>d1d6e18b3b42f9e445c8c6f….
<witnessScript> OP_2 <PubKeyA> <PubKeyB> <PubKeyC> OP_3 OP_CHECKMULTISIG

Transaction validated!!

Bitcoin Transaction

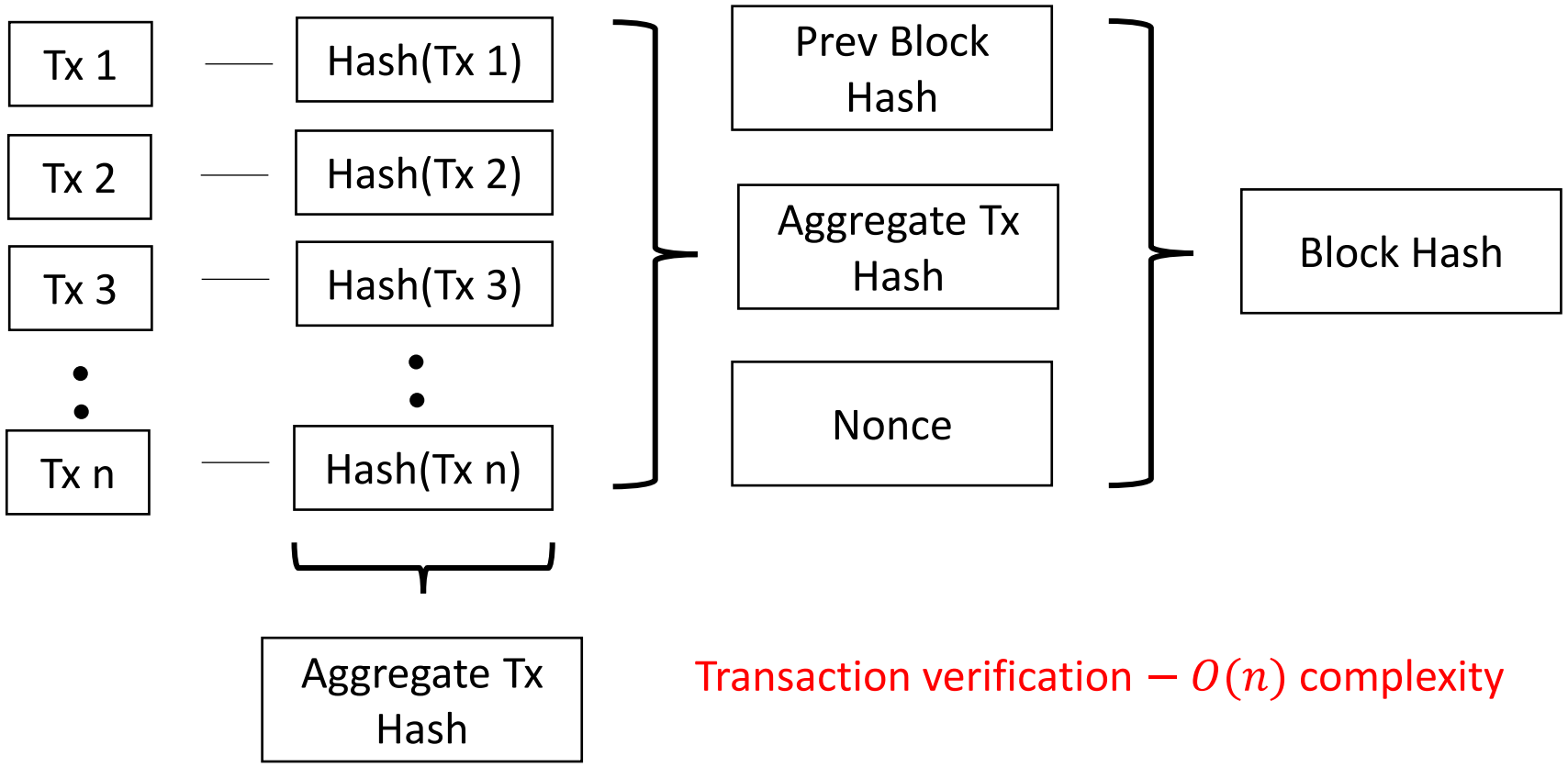
- How can you verify the correctness of a transaction, included in some block?
  - e.g., a vendor would like to verify the following customer transaction before delivering the merchandize
    - TXID: 7b9ab087c86818551aa77f59bb2c35bd774abed7fcb19c7e840b502b2604271c
    - Block No. 720,518
  - **Verification Steps**
    1. **Check Inclusion**
       - Confirm that the transaction with this TXID is part of Block No. 720,518
    2. **Validate Block Integrity**
       - Ensure this transaction's hash contributes to the overall block hash
       - This guarantees that if the transaction were missing or altered, the block hash would no longer match
    3. **Confirm Block Validity**
       - Verify that Block No. 720,518 itself is valid and part of the longest chain

Block Hash


Tx 1
Tx 2
Tx 3
Tx n
Hash(Tx 1)
Hash(Tx 2)
Hash(Tx 3)
Hash(Tx n)
Prev Block Hash
Aggregate Tx Hash
Nonce
Block Hash

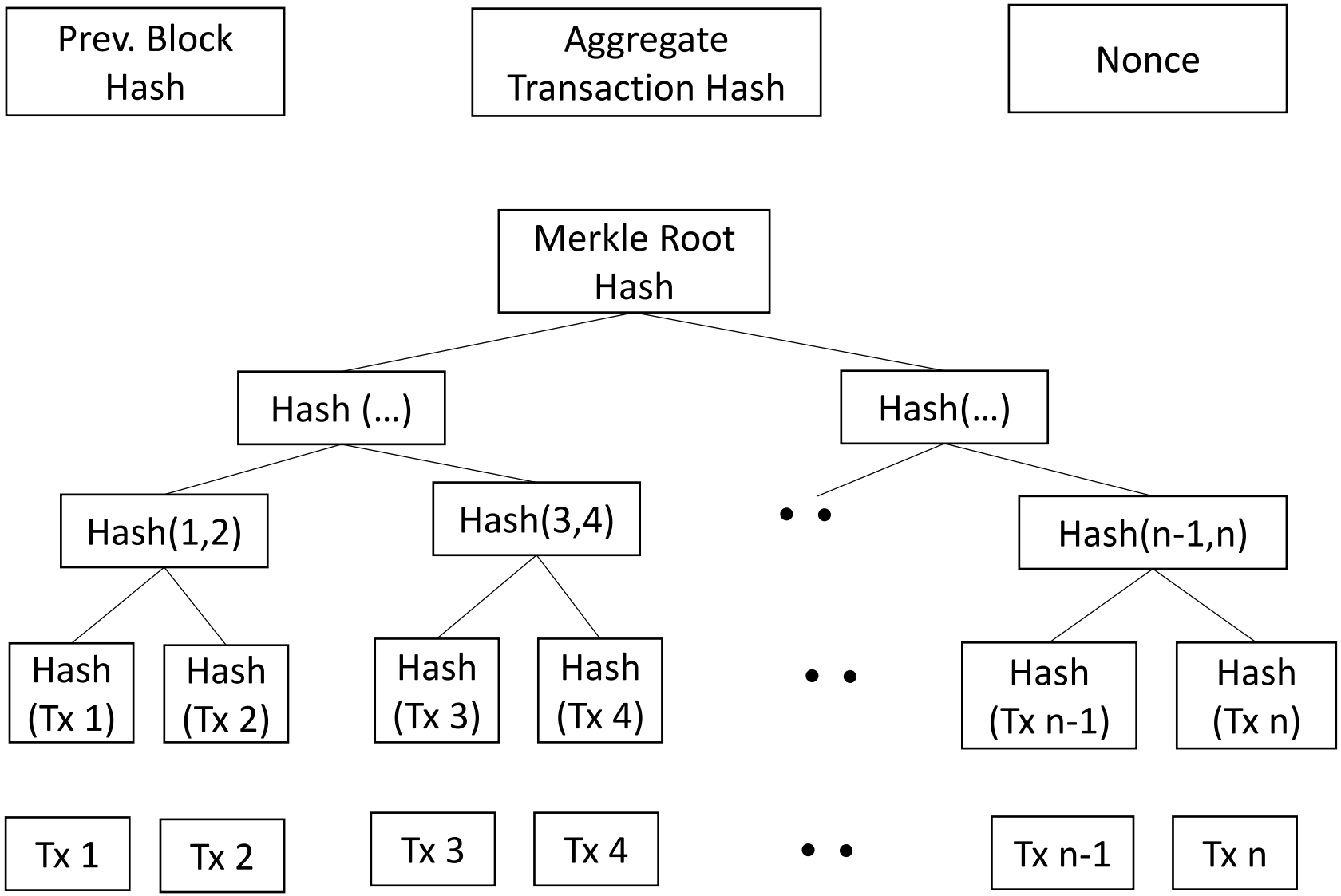
Block Hash

- Verify that some transaction, say Tx 2, is included in some block



Transaction verification $-\ O(n)$ complexity

Block Hash – Merkle tree



Block Hash – Merkle tree

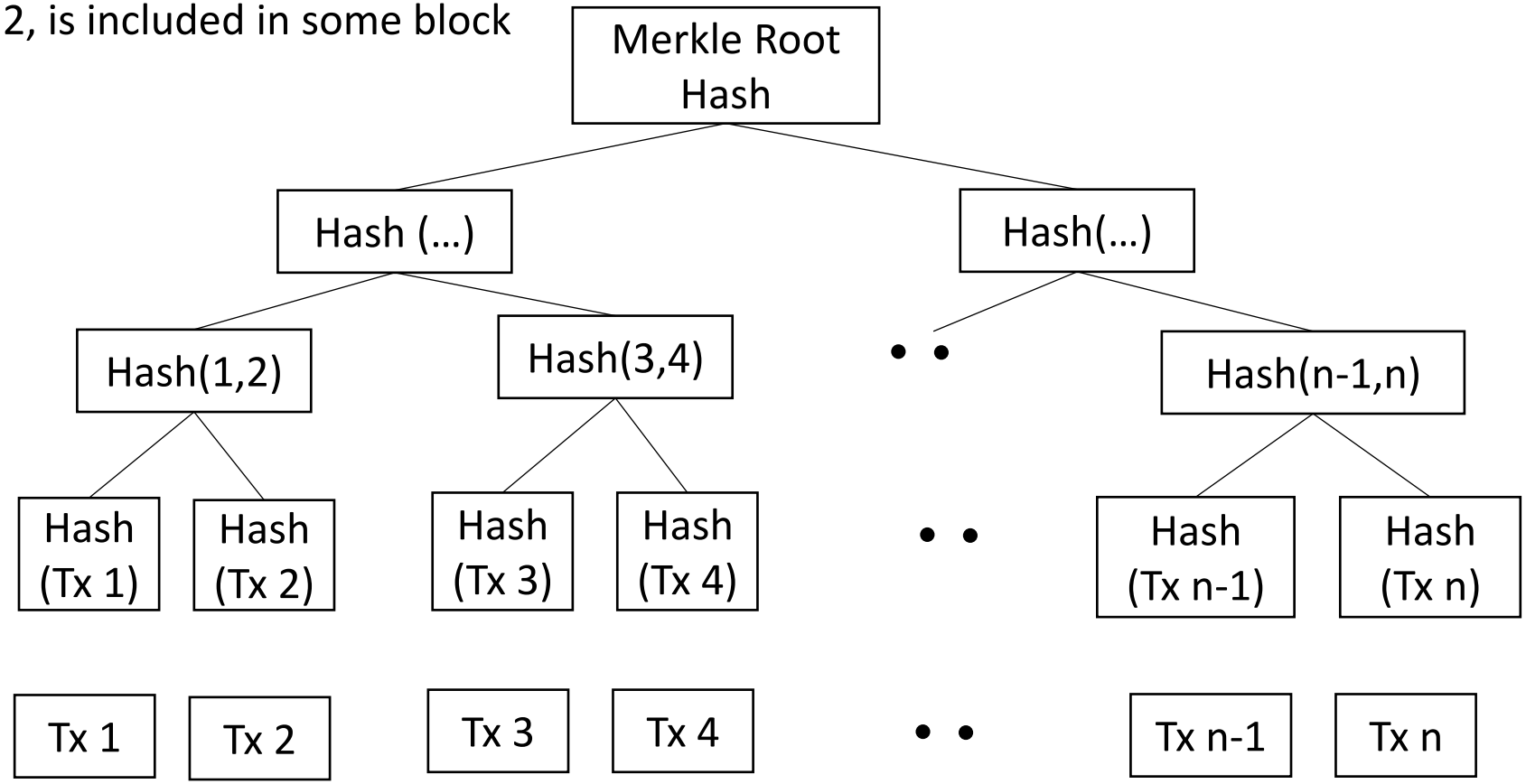
Prev. Block Hash

Aggregate Transaction Hash

Nonce

$O(\lg n)$

Verify that some transaction, say Tx 2, is included in some block



Merkle Tree

- **Summarizes transactions**
  - All transactions in a block are combined into one hash (the **Merkle root**) stored in the block header.
- **Enables lightweight verification (SPV)**
  - Clients can verify a transaction is in a block using a **Merkle proof** (only $\log_2(N)$ hashes, not the whole block).
- **Detects tampering**
  - Changing even one transaction alters the Merkle root → invalidates the block.
- **Improves scalability**
  - Proof size is small ($\approx \log_2(N)$), even if the block has thousands of transactions.